This code defines a method named define_method_attribute. This method will ultimately become a class method of ActiveRecord::Base, thanks to the mechanism we discussed in Chapter 10, *Active Support's Concern Module*, on page 179. Here, however, comes a twist: define_method_attribute is defined differently depending on the result of the Module.methods_transplantable? method.

Module.methods_transplantable? comes from the Active Support library, and it answers one very specific question: can I bind an UnboundMethod to an object of a different class? In *Unbound Methods*, on page 94, I mentioned that you can only do that from Ruby 2.0 onward, so this code defines define_method_attribute in two different ways depending on whether you're running Rails on Ruby 1.9 or 2.x.

Assume that you're running Ruby 2.0 or later. In this case, define_method_attribute retrieves an UnboundMethod from a cache of methods, and it binds the method to the current module with define_method. The cache of methods is stored in a constant named ReaderMethodCache.

(The call to generated_attribute_methods might look confusing—it returns a *Clean Room (87)* that serializes method definitions happening in different threads.)

Let's go see how ReaderMethodCache is initialized. The long comment gives an idea of how tricky it must have been to write this code:

gems/activerecord-4.1.0/lib/active_record/attribute_methods/read.rb

```
module ActiveRecord
  module AttributeMethods
    module Read
      ReaderMethodCache = Class.new(AttributeMethodCache) {
        private
        # We want to generate the methods via module_eval rather than
        # define_method, because define_method is slower on dispatch.
        # Evaluating many similar methods may use more memory as the instruction
        # sequences are duplicated and cached (in MRI).  define_method may
        # be slower on dispatch, but if you're careful about the closure
        # created, then define_method will consume much less memory.
        #
        # But sometimes the database might return columns with
        # characters that are not allowed in normal method names (like
        # 'my_column(omg)'. So to work around this we first define with
        # the __temp__ identifier, and then use alias method to rename
        # it to what we want.
        #
        # We are also defining a constant to hold the frozen string of
        # the attribute name. Using a constant means that we do not have
        # to allocate an object on each call to the attribute method.
        # Making it frozen means that it doesn't get duped when used to
        # key the @attributes_cache in read_attribute.
```